GUIDE 15 · ACCESS

Create a role and stamp out several keys from it

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=15-create-a-role. This copy was made on 4 November 2026.

Time: 5 minutes. **You'll end up with:** a named template of grants - a **role** - and several API keys created from it, plus a clear picture of what happens to those keys when the role changes later. (Short answer: nothing.)

A role carries the same grant fields a key does - `connections`, `queries`, `collections`, `allow_writes`, `allowed_write_ops`, `allowed_tables`, `rate_limit`, `allowed_ips` - and nothing else. It never authenticates anything itself; it exists so you don't type the same grants by hand for every partner, team or service.

Step 1: Create the role

shell

```
curl -X POST http://127.0.0.1:5000/api/v1/roles -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "partner", "connections": [], "queries": ["orders_by_status"], "rate_limit": "100/minute"
}'
```

json

```
{"name": "partner", "connections": [], "queries": ["orders_by_status"], "collections": [],
 "allow_writes": false, "allowed_write_ops": null, "allowed_tables": null, "rate_limit": "100/minute",
 "allowed_ips": null, "created_at": "2026-10-04 18:48:42", "example": false, "keys_created": 0}
```

(`orders_by_status` is the query from Give an external partner access to one query.)

In the Console: **Access** → **Roles** → **New role**.

Step 2: Create keys from it

shell

```
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "acme", "role": "partner"}'
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "globex", "role": "partner", "expires_at": "2026-12-31"}'
```

json

```
{"name": "acme", "connections": [], "queries": ["orders_by_status"], "rate_limit": "100/minute", ...,
 "created_from_role": "partner", "secret": "sk_..."}
```

Each key gets a **copy** of the role's grants and its own secret. `expires_at` is the one field you may add next to `role` - an expiry belongs to a key, not to a template (see Give a key an expiry date). Any grant field next to `role` is refused rather than silently merged:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "initech", "role": "partner", "rate_limit": "10/minute"}'
# {"error": "Cannot combine 'role' with explicit rate_limit - create the key from the role, then update
it
#  afterward to customize.", "code": "invalid_body", ...}
```

So to make one key a little different, create it from the role, then `PATCH /api/v1/api-keys/<name>` it.

In the Console, the Roles table's **New key from this** opens the new-key form with the role already chosen.

Step 3: Change the role - and see that the keys don't

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/roles/partner -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"rate_limit": "500/minute", "queries": ["orders_by_status", "salary_list"]}'

curl http://127.0.0.1:5000/api/v1/api-keys/acme -H 'X-API-Key: demo-key'
# {"name": "acme", "queries": ["orders_by_status"], "rate_limit": "100/minute", "created_from_role":
"partner", ...}
```

Verified: `acme` still has one query and `100/minute`. **A role is a template, not a live link.** Requests are checked against the key's own grants only; the role is never consulted again after the key is created. Deleting the role (`DELETE /api/v1/roles/partner`) leaves `acme` and `globex` exactly as they were, `created_from_role` included - it is a label for you, not something any permission check reads.

That's deliberate: editing a role can never widen (or break) access for keys already handed out. The flip side is that rolling a change out to existing keys means patching each key. The role's `keys_created` count, and the API keys table's "from role" note, tell you which keys came from it.

The one exception is signed-in users (JWT): they have no stored key, so they use their role live - a role change applies to them on their next request.

What the audit log records

Every create, update and delete of a role is audited, with a diff for an update:

```shell
curl 'http://127.0.0.1:5000/api/v1/audit?target=partner' -H 'X-API-Key: demo-key'
# {"items": [..., {"action": "update_role", "target": "partner",
#   "changes": {"queries": {"from": ["orders_by_status"], "to": ["orders_by_status", "salary_list"]},
#               "rate_limit": {"from": "100/minute", "to": "500/minute"}}}, ...]}
```

Next steps

- Set up your first scoped API key - every grant field a role can carry.
- Read the audit log - who changed which role, and when.

- The built-in example APIs load five roles with one key each - a quick way to see roles in the Console before making your own.